TECHNISCHE UNIVERSITÄT WIEN

Files, Shared Memory, Semaphores

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch,
Peter Puschner
2026-10-20

Part I

Files

File Descriptors

- Process keeps a table of open files
- The index of an entry in the table is called file descriptor
- Non-negative integer
- Standard I/O file descriptors: already present at program start

fd	Description	POSIX[1] Name	`stdio.h` Stream Name
0	Standard input	`STDIN_FILENO`	`stdin`
1	Standard output	`STDOUT_FILENO`	`stdout`
2	Error output	`STDERR_FILENO`	`stderr`

- POSIX names are defined in <`unistd.h`>
- System calls for file handling are defined in <`fcntl.h`>

[1] Portable Operating System Interface: a standardization of Unix-like OSs

Opening (and possibly creating) a File: open(2)


```c
int open(const char *pathname, int flags, mode_t mode);     /* Function prototype */
int fd = open("/data.txt", O_CREAT | O_EXCL | O_WRONLY);    /* Sample call */
```

- pathname: path to the file
- flags: combination of flags (bitwise OR). Examples:
  - O_RDONLY, O_WRONLY, O_RDWR: read only, write only, or both (one of these must be present)
  - O_APPEND: append instead of overwriting (if file exists)
  - O_CREAT: create the file if it does not exist (default)
  - O_EXCL: fail if the file already exists
- mode: permissions to apply when a new file is created
- Returns a file descriptor, or -1 on error ($\rightarrow$ errno)

Reading from a File: read(2)


```
1  ssize_t read(int fd, void *buf, size_t count);    /* Function prototype */
```

- fd: file descriptor
- buf: buffer to be filled with the read data
- count: size of buffer (max number of bytes to read)
- Returns number of bytes effectively read, or -1 on error ($\rightarrow$ errno)

```
1  char buffer[80];
2  for (int pos = 0; pos < sizeof(buffer); ) {
3      int numread = read(fd, buffer+pos, sizeof(buffer)-pos);
4
5      if (numread < 0) { /* error */ }
6      else if (numread == 0) { break; /* end of file */ }
7      else { pos += numread; }
8  }
```

Writing to a File: `write(2)`


```
1  ssize_t write(int fd, const void *buf, size_t count);    /* Function prototype */
```

- fd: file descriptor
- buf: buffer with the data to be written
- count: size of buffer (max number of bytes to write)
- Returns the number of bytes effectively written, or -1 on error ($\rightarrow$ `errno`)

```
1  char buffer[80] = "Data to be written";
2  for (int pos = 0; pos < sizeof(buffer); ) {
3      int n = write(fd, buffer+pos, sizeof(buffer)-pos);
4
5      if (n < 0) { /* error */ }
6      else { pos += n; }
7  }
```

Closing a File: `close(2)`


```c
int close(int fd);    /* Function prototype */
```

- fd: file descriptor
- Returns 0 on success and -1 on error ($\rightarrow$ `errno`)

```c
if (close(fd) < 0) {
    fprintf(stderr, "close failed: %s\n", strerror(errno));
    exit(EXIT_FAILURE);
}
```

Dealing with EINTR

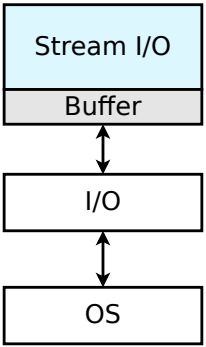
- read() and write() can be interrupted by a signal
- In this case they return -1 and set `errno` to `EINTR`
- No bytes are read or written: just try again

```
1  char buffer[80];
2  for (int pos = 0; pos < sizeof(buffer); ) {
3      int numread = read(fd, buffer+pos, sizeof(buffer)-pos);
4
5      if (numread < 0) {
6        if (errno == EINTR) { continue; }
7        /* deal with error */
8      }
9      else if (numread == 0) { break; /* end of file */ }
10     else { pos += numread; }
11 }
```

Stream I/O in C


```c
#include <stdio.h>
```

- Buffered wrapper for Unix I/O system calls
- Stream data type: `FILE`. Includes:
  - File descriptor
  - Pointer to buffer
  - Current position,
  - EOF and error flags
- Predefined streams: `stdin`, `stdout`, `stderr`
- Convention: functions start with “f”
  (`fopen(3)`, `fdopen(3)`, `fwrite(3)`, `fprintf(3)`, ...)



Opening a Stream: `fopen(3)`

```
FILE *fopen(const char *path, const char *mode);
```

- path: Path to the file
- mode: one of:Different I/O modes:
  - `"r"` / `"w"` / `"a"` : read/write/append-only
  - `"r+"` / `"w+"` / `"a+"` : read and write (update mode)

  Note: `"w"` and `"w+"` truncate the file if it exists
- Returns a stream, or NULL on failure ($\rightarrow$ `errno`)

```
1  FILE *input = fopen("data.txt", "r");
2
3  if (input == NULL) {
4      fprintf(stderr, "fopen failed: %s\n", strerror(errno));
5      exit(EXIT_FAILURE);
6  }
```

File Descriptor to Stream: `fdopen(3)`


```
FILE *fdopen(int fd, const char *mode);
```

- Returns a stream that wraps file descriptor fd
- `mode`: I/O mode
- Returns a stream, or NULL on failure (→ `errno`)

```
1  int fd = open(...);
2  if (fd < 0) {
3      // error
4  }
5
6  FILE *f = fdopen(fd, "r");
7  if (f == NULL) {
8      // error
9  }
```

Function	Description
fread	Reads *n* elements, each *s* bytes long
fgets	Reads a line (up to `'\n'`)
fgetc	Reads a character
fwrite	Writes *n* elements, each *s* bytes long
fputs	Writes a C-string
fputc	Writes a character
fprintf	Formatted printing
fseek	Set the file position indicator
Since POSIX.1-2008:	
getline	Reads a line into a dynamically allocated buffer

Stream Status


```
int ferror(FILE *stream);
int feof(FILE *stream);
void clearerr(FILE *stream);
int fileno(FILE *stream);
```

- `ferror(2)` tests the error indicator of the stream (0 = no errors).
- `feof(2)` tests if the stream reached end-of-file (0 = not reached).
- `clearerr(2)` resets error and end-of-file indicators.
- `fileno(2)` returns the file descriptor of a stream.
  Example: `fileno(stdout)` → `1`

Flushing and Closing Streams: `fflush(3)` and `fclose(3)`


```
int fflush(FILE *stream);
int fclose(FILE *stream);
```

- `fflush(3)` force writing of buffered data to the file
- `fclose(3)` flushes the stream and closes it (along with the fd)

Both return 0 on success, `EOF` on failure ($\rightarrow$ `errno`)
(`EOF` is also defined in `stdio.h`, usually as -1)

Example: Read and Write with Streams

Read the content of `input.txt` line by line and write it to `output.txt`

```c
char buffer[1024];
FILE *in, *out;

if ((in = fopen("input.txt", "r")) == NULL) { /* fopen failed */ }
if ((out = fopen("output.txt", "w")) == NULL) { /* fopen failed */ }

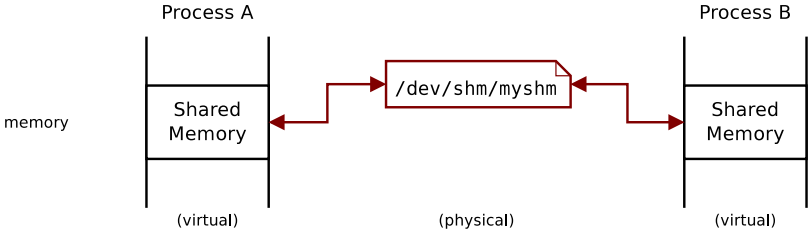
while (fgets(buffer, sizeof(buffer), in) != NULL) {
    if (fputs(buffer, out) == EOF) { /* fputs failed */ }
}
if (ferror(in)) { /* fgets failed */ }

fclose(in);
fclose(out);
```

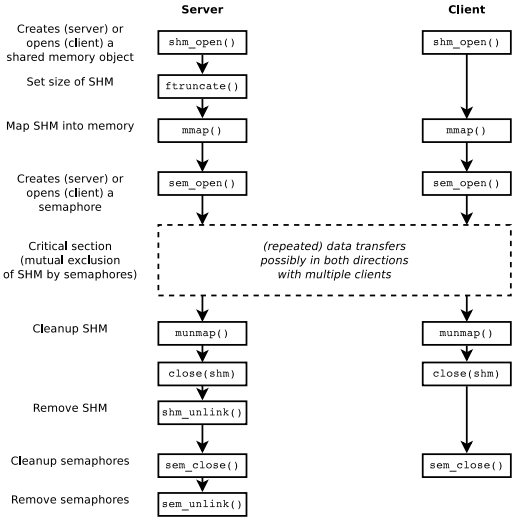
Part II

Shared Memory

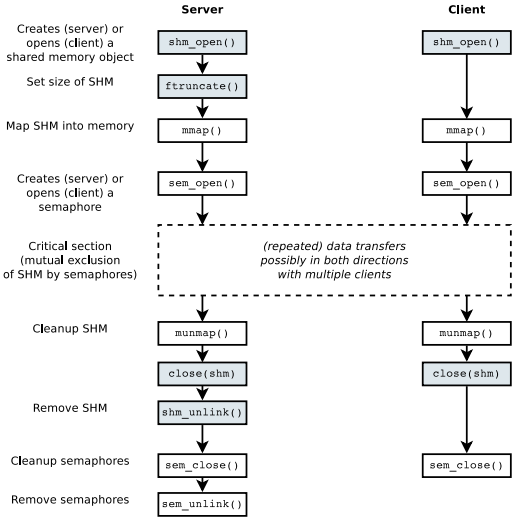
- Region of memory that may be accessed by multiple processes
- The region may be mapped into the address space of each process
- Very fast inter-process communication (IPC)
- Use normal operations to read and modify (Streams)
- Processes must synchronize to avoid simultaneous access



Client-Server Example



Client-Server Example



Create/Open a Shared Memory Object: `shm_open(3)`


```c
#include <sys/mman.h>
#include <fcntl.h>     /* For O_* constants */

int shm_open(const char *name, int oflag, mode_t mode);
```

- `name`: Name for the object, like "/somename":
  An initial slash, followed by one or more characters, none of which are slashes.
- `oflag`: Bit mask: O_RDONLY or O_RDWR plus optional flags:
  - O_CREAT: creates an object unless it exists
  - additionally O_EXCL: error if already exists
  - O_TRUNC: if already exists, truncate it (= reset its length to 0)
- `mode`: If object must be created, sets access rights.
  - Mandatory when O_CREAT in `oflag`
  - Optional (and ignored) otherwise
- Return value: file descriptor on success, -1 on error ($\rightarrow$ errno)
- Linux creates object at `/dev/shm/somename`

Set Size: `ftruncate(2)`


```
1 #include <unistd.h>
2 #include <sys/types.h> /* for off_t */
3
4 int ftruncate(int fd, off_t length);
```

- Typically done by the process that creates the shared memory object
- Return value: 0 on success, -1 on error ($\rightarrow$ errno)
- (Note: works on any file descriptor, not only on shared memory objects)

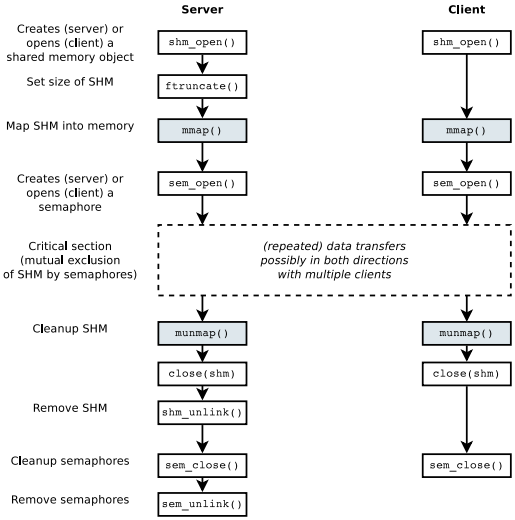
Now the fd may be memory-mapped (more on that later)

Remove a Shared Memory Object: `shm_unlink(3)`


```c
int shm_unlink(const char *name);
```

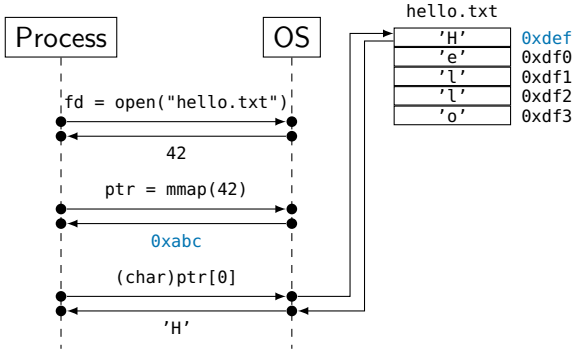
- `name`: object name (from `shm_open()`)
- Returns 0 on success, -1 on error ($\rightarrow$ errno)
- Further `shm_open()` with the same name raise an error
  (until a new object is created with the same name)
- Memory is still accessible to processes that have already opened the object
- The memory is only released when every process has closed/unmapped it
  (`close()`/`munmap()`)
- Shell commands (`ls`, `rm`) can be used to list and remove `/dev/shm/name`
  (e.g. if program crashes)

Client-Server Example



Memory Mapping

- Done with `mmap(2)`
  - Takes a file descriptor
  - Returns a `void *`
  - (Casting is developer's responsibility)
- Pointer contains a virtual address; OS takes care of converting it to/from a physical address
- Mappings have a name; in Linux they are in `/proc/pid/maps`



Mapping a File to Memory: `mmap(2)`


```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

- addr Suggestion for starting address, leave `NULL`
- length Size of the mapping in bytes
- prot Bit mask for memory protection: PROT_NONE (no access allowed), PROT_READ, PROT_WRITE
- flags Bit mask, e.g., MAP_PRIVATE, MAP_SHARED, MAP_ANONYMOUS
- fd The file descriptor to be mapped
- offset Offset in the file (in pages), usually 0

Returns an address, or `MAP_FAILED` on error (errno)

Releasing a Mapping: `munmap(2)`


```c
#include <sys/mman.h>
int munmap(void *addr, size_t length);
```

- Unmaps at least `length` bytes, starting at `addr`
- Why "at least"? Because most OSs handle mapping in pages, not bytes
- Return value: 0 on success, -1 on error ($\rightarrow$ errno)

Example

Define structure of shared memory

```c
#include <fcntl.h>
#include <stdio.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <unistd.h>

#define SHM_NAME "/myshm"
#define MAX_DATA (50)

struct myshm {
  unsigned int state;
  unsigned int data[MAX_DATA];
};
```

Example

Create and map the shared memory

```
1  // create and/or open the shared memory object:
2  int shmfd = shm_open(SHM_NAME, O_RDWR | O_CREAT, 0600);
3  if (shmfd == -1) { /* error */ }
4
5  // set the size of the shared memory:
6  if (ftruncate(shmfd, sizeof(struct myshm)) < 0) { /* error */ }
7
8  // map shared memory object:
9  struct myshm *myshm;
10 myshm = mmap(NULL, sizeof(*myshm), PROT_READ | PROT_WRITE, MAP_SHARED, shmfd, 0);
11 if (myshm == MAP_FAILED) { /* error */ }
12
13 // Notice that the file descriptor can be closed after mmap
14 if (close(shmfd) == -1) { /* error */ }
```

Usage and Cleanup

- Usage: just access `myshm->state` and `myshm->data[]`
- But we will need additional safeguards if multiple processes want to do that concurrently (more on that later)
- Cleanup:

```
1 // unmap shared memory:
2 if (munmap(myshm, sizeof(*myshm)) == -1) { /* error */ }
3
4 // remove shared memory object (only for the creator):
5 if (shm_unlink(SHM_NAME) == -1) { /* error */ }
```

Part III

Semaphores

Mutual Exclusion Problem

- Block of code that accesses shared resources (e.g., shared memory)
- Only one process/thread may be in critical section at a time

Example: two threads working on a shared variable `int *x`, initially `NULL`:

```
// Thread A:
a1: if (x == NULL)
a2:   x = malloc(sizeof(int));
```

```
// Thread B:
b1: if (x == NULL)
b2:   x = malloc(sizeof(int));
```

- "Good" executions: a1a2b1, b1b2a1
- "Bad" executions: a1b1a2b2, ...
- In the bad execution, x is allocated twice!
- The first memory address is lost → memory leak

Think of a semaphore as a shared counter with 3 basic, atomic operations:

- S = Init(N)
  create semaphore S with value N
- P(S)[2], aka Wait(S), Down(S)
  decrement S and block when S gets ≤ 0
- V(S)[3], aka Post(S), Signal(S), Up(S)
  increment S and wake up one blocked process

[2] From "passering" or "prolaag (probeer te verlagen)"
[3] From "vrijgave" or "verhoog". Dijkstra was Dutch...

Example - Mutex


```
// Thread A:
a1: if (x == NULL)
a2:   x = malloc(sizeof(int));
```

```
// Thread B:
b1: if (x == NULL)
b2:   x = malloc(sizeof(int));
```

How to guarantee that only one thread is entering the critical section?

Example - Mutex

Initialization:

```
1  mutex = Init(1)
```

Thread A:

```
1  P(mutex) // wait
2  if (x == NULL)
3    x = malloc(sizeof(int));
4  V(mutex) // post
```

Thread B:

```
1  P(mutex) // wait
2  if (x == NULL)
3    x = malloc(sizeof(int));
4  V(mutex) // post
```

Example - Alternating Execution

Thread A:

```
1  for(;;) {
2    (shm->state)++;
3  }
```

Thread B:

```
1  for(;;) {
2    (shm->state)++;
3  }
```

How to achieve that A and B are called alternately?

- Notice that assignments to “basic types” is not atomic!
- First, new value is evaluated on current state
- Then, new value is written back to memory
- Another thread/process may change the state in between!

Example - Alternating Execution

Initialization:

```
1   S1 = Init(1)
2   S2 = Init(0)
```

Thread A:

```
1   for(;;) {
2     P(S1) // wait
3     x = x + 1
4     V(S2) // post
5   }
```

Thread B:

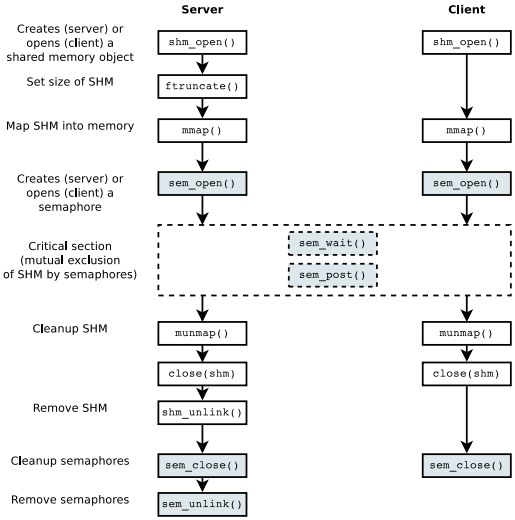
```
1   for(;;) {
2     P(S2) // wait
3     x = x + 1
4     V(S1) // post
5   }
```

S1, S2 force A, B to take turns

POSIX Named Semaphores

- Similar to POSIX shared memory
  - Identified by name
  - Created on dedicated file system for volatile memory: tmpfs
  - Lifetime limited to system runtime
- Must use `gcc -pthread` (and possibly `-lrt`) for linking
- Linux: object is created at `/dev/shm/sem.somename`
- Documentation: `man 7 sem_overview`

Client-Server Example



Create/Open a Semaphore: sem_open(3)


```
1 #include <semaphore.h>
2 #include <fcntl.h>     /* For O_* constants */
3 /* create a new named semaphore */
4 sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
5 /* open an existing named semaphore */
6 sem_t *sem_open(const char *name, int oflag);
```

name Name of the form "/somename"

oflag Bit mask: O_CREAT, O_EXCL

mode Access permissions[4]

value Initial value[4]

Returns the semaphore's address on success, SEM_FAILED on error ($\rightarrow$ errno)

[4] Only used if O_CREAT and semaphore did not exist

Close and Remove Semaphores: `sem_close(3)`, `sem_unlink(3)`


```c
int sem_close(sem_t *sem);
int sem_unlink(const char *name);
```

- `sem_unlink(3)` removes the named semaphore, but only after it has been closed by all processes
- Return value for both: 0 on success, -1 on error ($\rightarrow$ errno)

sem_wait(3)


```
int sem_wait(sem_t *sem);
```

- If the value > 0, the method decrements it and returns immediately
- Otherwise, it blocks the caller until the value gets positive
- Return value: 0 on success, -1 on error ($\rightarrow$ errno)
- On error, the value of the semaphore is not changed

Signal Handling

The function `sem_wait()` can be interrupted by a signal (errno == EINTR)!

sem_post(3)


```
int sem_post(sem_t *sem);
```

- Increments the value of the semaphore
- If the value becomes positive, a blocked process will continue
- If multiple processes are waiting: the order is not defined (= weak semaphore)
- Return value: 0 on success, -1 on error ($\rightarrow$ errno)
- On error, the value of the semaphore is not changed

Example - Alternating Execution

Process A (code without error handling)

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <semaphore.h>
4  #include <fcntl.h>
5
6  #define SEM_1   "/sem_1"
7  #define SEM_2   "/sem_2"
8  int main(int argc, char **argv) {
9    sem_t *s1 = sem_open(SEM_1, O_CREAT | O_EXCL, 0600, 1);
10   sem_t *s2 = sem_open(SEM_2, O_CREAT | O_EXCL, 0600, 0);
11   for(int i = 0; i < 3; ++i) {    sem_wait(s1);
12                                   printf("critical: %s: i = %d\n", argv[0], i);
13                                   sem_post(s2); sleep(1); // do not sleep in critical section
14                             }
15   sem_close(s1); sem_close(s2);
16   sem_unlink(SEM_1); sem_unlink(SEM_2);
17   return 0;
18 }
```

Example - Alternating Execution

Process B (code without error handling)

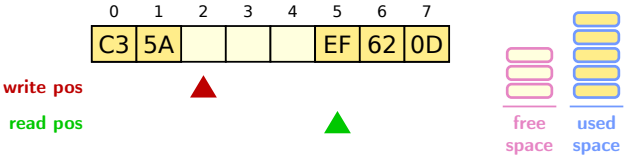
```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <semaphore.h>
4  #include <fcntl.h>
5
6  #define SEM_1   "/sem_1"
7  #define SEM_2   "/sem_2"
8  int main(int argc, char **argv) {
9    sem_t *s1 = sem_open(SEM_1, 0);
10   sem_t *s2 = sem_open(SEM_2, 0);
11   for(int i = 0; i < 3; ++i) {    sem_wait(s2);
12                                   printf("critical: %s: i = %d\n", argv[0], i);
13                                   sem_post(s1); sleep(1);
14                               }
15   sem_close(s1); sem_close(s2);
16   return 0;
17 }
```

Part IV

Extra slides - Circular Buffer

Circular Buffer

= simple FIFO implementation with shared memory and semaphores




```
1  int wr_pos = 0;
2  sem_t *free; // to BUF_LEN
3  sem_t *used; // to 0
4  void write(int val) {
5     sem_wait(free);
6     buf[wr_pos] = val;
7     sem_post(used);
8     wr_pos += 1;
9     wr_pos %= sizeof(buf);
10 }
```

```
1  int rd_pos = 0, LEN = 8;
2  sem_t *free; // to BUF_LEN
3  sem_t *used; // to 0
4  int read() {
5   sem_wait(used);
6   int val = buf[rd_pos];
7   sem_post(free);
8   rd_pos = (rd_pos+1) % LEN;
9   return val;
10 }
```

Exercise 1

Producer/consumer example using a circular buffer

- Producer(s) write(s) data to the circular buffer
- Consumer reads from the circular buffer
- Synchronization using semaphores

Summary

- Shared memory is a fast method for IPC
- Explicit synchronization with semaphores
- Synchronization tasks
- Strategies to resource (de-)allocation

- Michael Kerrisk: A Linux and UNIX System Programming Handbook, No Starch Press, 2010.
- Linux implementation of shared memory/tmpfs:
  `http://www.technovelty.org/linux/shared-memory.html`
- Richard W. Stevens: UNIX Network Programming,
  Vol. 2: Interprocess Communications